

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Facultad de Ingeniería

ACTIVIDAD 1 — CATÁLOGO EN PHP 8 CON POO Y JSON

Tienda virtual de ropa «Manos del Oriente»

Aplicación de catálogo en PHP 8.4 con programación orientada a objetos, formularios validados y persistencia en JSON

Integrantes:

1. Daniela Cayo Mamani
2. Nombre del integrante 2
3. Nombre del integrante 3

Carrera: Ingeniería de Sistemas

Asignatura: Tecnología Web II

Docente: Nombre del docente

Santa Cruz de la Sierra, Bolivia

12 de octubre de 2026

Índice

Resumen 4
Introducción 5
Marco teórico 7
Programación orientada a objetos en PHP 8 7
Validación, saneamiento y seguridad 7
HTTP y JSON 7
Metodología 9
Resultados 10
Paso 1. Alcance y backlog 10
Paso 2. Requisitos 10
Paso 3. Arquitectura y organización del código 11
Paso 4. Modelo orientado a objetos 12
Paso 5. Interfaz y formularios 13
Paso 6. Subida segura de la foto y seguridad adicional 15
Paso 7. Pruebas automáticas 16
Discusión 20
Conclusiones 21
Referencias 22
Anexos 23
Anexo A. Registro de uso de inteligencia artificial 23
Anexo B. Matriz de trazabilidad de la Actividad 1 24
Anexo C. Glosario 24

Índice de figuras y tablas

Tabla 1. Alcance de la Actividad 1: dentro y fuera 10
Tabla 2. Backlog de la Actividad 1 en el tablero de GitHub Projects 10
Tabla 3. Requisitos del catálogo de la tienda 11
Tabla 4. Estructura de carpetas del proyecto 11
Figura 1. Constructor de la clase Producto con promoción de propiedades 12
Figura 2. Vitrina de la tienda con los datos de demostración 14
Figura 3. Formulario con errores de validación por campo 15
Figura 4. Página de gestión del catálogo 16
Tabla 5. Resumen de las comprobaciones automáticas 17
Figura 5. Ejecución de las pruebas automáticas 18
Tabla 6. Cálculo manual del valor del inventario 19
Tabla 7. Registro de uso de IA en la Actividad 1 23
Tabla 8. Matriz de trazabilidad: requisito, issue, implementación y prueba 24

Resumen

Este informe presenta la primera actividad del proyecto «Manos del Oriente», el sistema web de la Asociación de Confeccionistas Textiles del mismo nombre, un caso verosímil ubicado en la Chiquitanía (Santa Cruz, Bolivia) que sirve de base al proyecto final de la asignatura Tecnología Web II. Se desarrolló en PHP 8.4, con tipado estricto, la aplicación de catálogo de la tienda virtual de ropa: una vitrina con filtros, la ficha de cada prenda y la gestión para registrar, editar y eliminar prendas con su foto. Se modelaron las clases Producto, Categoria y Catalogo con promoción de propiedades en el constructor y propiedades de solo lectura, además de una enumeración para las tallas, y los datos se guardaron en archivos JSON. Los formularios se validan y sanean en el servidor, están protegidos contra la falsificación de solicitudes (CSRF) y la subida de fotos sigue las recomendaciones de OWASP. La verificación se realizó con 33 comprobaciones automáticas que ejecutan peticiones HTTP reales contra el servidor embebido de PHP, todas satisfactorias. Se documenta también el uso de inteligencia artificial, incluidos dos errores de la IA que las pruebas pusieron en evidencia.

Palabras clave: PHP 8, programación orientada a objetos, validación de formularios, JSON, comercio electrónico, seguridad web

Introducción

Las pequeñas asociaciones productivas de Bolivia suelen vender en ferias y por mensajería instantánea, sin un catálogo actualizado que muestre precios, tallas y existencias. La Asociación de Confeccionistas Textiles «Manos del Oriente» agrupa a costureras y bordadoras de la Chiquitanía que confeccionan poleras, pantalones, vestidos, abrigos y accesorios con telas y técnicas de la región, como el bordado en punto cruz y el tejido de saó. Hoy registra su inventario en cuadernos y comparte fotos por WhatsApp, lo que provoca precios desactualizados, ventas de prendas agotadas y la dificultad de saber cuánto vale el inventario.

El proyecto final de Tecnología Web II propone desarrollar y publicar el sistema web de una asociación real o verosímil, y esta actividad construye su primera pieza: el catálogo de la tienda virtual. El objetivo fue desarrollar en PHP 8 una aplicación de catálogo con programación orientada a objetos (POO), formularios validados y persistencia en JSON, y demostrar que todas las pruebas pasan. Los objetivos específicos fueron: (a) configurar el entorno con PHP 8.4, el servidor embebido y declare(strict_types=1); (b) escribir funciones tipadas con parámetros nombrados y valores por defecto, organizadas con include y require; (c) implementar formularios GET y POST con validación, saneamiento y subida segura de la foto; (d) modelar las clases Producto, Categoria y Catalogo con promoción de propiedades y persistirlas en JSON; y (e) registrar el uso de inteligencia artificial en una bitácora.

El documento se organiza así: el marco teórico reúne los conceptos que sostienen las decisiones; la metodología describe cómo se organizó y verificó el trabajo; los resultados presentan el alcance, los requisitos, la arquitectura, el modelo de clases, la interfaz, la seguridad y las pruebas; la discusión interpreta los hallazgos; y las conclusiones cierran con el trabajo

futuro. Los anexos contienen el registro de uso de inteligencia artificial, la matriz de trazabilidad y un glosario.

Marco teórico

Programación orientada a objetos en PHP 8

PHP 8.0 introdujo la promoción de propiedades en el constructor, que permite declarar y asignar una propiedad directamente en la lista de parámetros (The PHP Group, s.f.-a). PHP 8.1 agregó las propiedades de solo lectura (readonly) y las enumeraciones, y PHP 8.2, las clases readonly (The PHP Group, s.f.-b). Estas características reducen el código repetitivo y permiten crear objetos inmutables, cuyo estado no cambia después de construirse, lo que facilita razonar sobre ellos (Lockhart, 2015).

Validación, saneamiento y seguridad

Toda entrada que llega del navegador debe tratarse como no confiable. Validar significa rechazar los datos que no cumplen las reglas; sanear significa normalizarlos, por ejemplo quitando espacios o etiquetas HTML. La subida de archivos es una de las funciones más riesgosas de una aplicación web: el nombre y el tipo declarados por el navegador pueden falsificarse, por lo que se recomienda verificar el contenido real, limitar el tamaño, generar un nombre nuevo y guardar el archivo fuera de la raíz pública (OWASP Foundation, s.f.-a). Para la falsificación de solicitudes entre sitios (CSRF), la defensa recomendada es un token secreto por sesión que el servidor compara en cada envío (OWASP Foundation, s.f.-b). Ambas categorías figuran entre los riesgos más críticos del OWASP Top 10 (OWASP Foundation, 2021).

HTTP y JSON

Los métodos GET deben ser seguros, es decir, no deben modificar el estado del servidor (Fielding et al., 2022). Por eso la búsqueda usa GET y las operaciones que cambian datos usan POST, seguidas de una redirección 303 (patrón Post/Redirect/Get) para evitar reenvíos

accidentales. JSON es un formato de texto liviano para el intercambio de datos (Bray, 2017) y resulta adecuado como persistencia inicial antes de migrar a una base de datos.

Metodología

El desarrollo partió de los laboratorios 1 a 4 de la asignatura, cuya estructura de carpetas y motor de pruebas (Verificador.php) se reutilizaron. Cada unidad de trabajo se registró como un issue en el tablero de sprint del proyecto en GitHub Projects (https://github.com/orgs/TecnologiaWeb2/projects/1), con las columnas Backlog, To Do, In Progress, In Review y Done, y con un hito por actividad. El código se versionó en https://github.com/TecnologiaWeb2/Tecnologiaweb2 y la versión entregada se marcó con la etiqueta actividad-1.

Se utilizó Claude Code (Anthropic, 2026) como asistente de programación con acceso al repositorio. La regla de trabajo fue que la IA propone y el equipo decide: cada propuesta se comparó con los laboratorios anteriores y su funcionamiento se confirmó ejecutando las pruebas. Se utilizó PHP 8.4.13 en Windows 11, Visual Studio Code como editor y el servidor embebido de PHP (php -S localhost:8000 -t public). Todos los archivos comienzan con declare(strict_types=1), de modo que PHP lanza un error si una función recibe un tipo distinto al declarado en lugar de convertirlo en silencio.

Las pruebas trabajan sobre una copia de los datos en una carpeta temporal, indicada con la variable de entorno TIENDA_DATOS, de modo que nunca modifican el catálogo real. Los valores esperados de los cálculos se verificaron a mano antes de escribirlos en las pruebas.

Resultados

Paso 1. Alcance y backlog

La Tabla 1 delimita qué se comprometió en esta actividad. Los módulos de base de datos, usuarios, pedidos y API quedan para las actividades siguientes del proyecto.

Tabla 1

Alcance de la Actividad 1: dentro y fuera

Dentro de la actividad	Fuera de la actividad
Vitrina con filtros GET, ficha de la prenda y error 404.	Base de datos MySQL (Actividad 2).
Registro, edición y eliminación de prendas con foto.	Gestión de socios como entidad propia (Actividad 2).
Clases Producto, Categoria y Catalogo con persistencia en JSON.	Usuarios, roles y pedidos (Actividad 3).
Validación, saneamiento, CSRF y subida segura de fotos.	API REST y consumo con fetch (Actividad 4).
Cálculo del valor del inventario.	Publicación en un hosting (proyecto final).

Nota. Elaboración propia. Cada elemento de la columna derecha figura como issue en el tablero del proyecto.

Tabla 2

Backlog de la Actividad 1 en el tablero de GitHub Projects

Issue	Unidad de trabajo	Requisito	Estado
#1	Estructura base del proyecto	RNF-01	Done
#2	Modelos POO: Producto, Categoria y enum Talla	RF-04	Done
#3	Catálogo con persistencia en JSON	RNF-04	Done
#4	Validación y saneamiento del formulario de prendas	RNF-01	Done
#5	Subida segura de fotos	RNF-02	Done
#6	Vitrina, ficha de producto y gestión de prendas	RF-01 a RF-05	Done
#7	Pruebas automáticas de la Actividad 1 (33/33)	Todos	Done
#8	Informe APA, presentación y capturas	—	Done

Nota. Elaboración propia a partir del tablero https://github.com/orgs/TecnologiaWeb2/projects/1.

Paso 2. Requisitos

A partir del caso se definieron los requisitos funcionales y no funcionales de la Tabla 3. Cada uno tiene al menos una prueba automática que lo verifica (Anexo B).

Tabla 3

Requisitos del catálogo de la tienda

Código	Requisito	Tipo
RF-01	Mostrar las prendas con foto, categoría, precio, tallas y disponibilidad	Funcional
RF-02	Filtrar por texto, categoría, talla y stock, y ordenar por nombre o precio	Funcional
RF-03	Ver la ficha de detalle de una prenda	Funcional
RF-04	Registrar, editar y eliminar prendas con su foto	Funcional
RF-05	Calcular el valor del inventario (precio × stock)	Funcional
RNF-01	Validar y sanear toda entrada en el servidor	Seguridad
RNF-02	Aceptar solo imágenes JPG, PNG o WebP de hasta 2 MB	Seguridad
RNF-03	Proteger los formularios POST contra CSRF	Seguridad
RNF-04	Persistir los datos en JSON sin corromper el archivo	Fiabilidad
RNF-05	Interfaz adaptable a celulares y accesible	Usabilidad

Nota. RF = requerimiento funcional; RNF = requerimiento no funcional.

Paso 3. Arquitectura y organización del código

El proyecto separa la carpeta pública, la única que expone el servidor, del código y los datos (Tabla 4). Cada página carga todo lo necesario con un único require de src/arranque.php, que a su vez usa require_once para las clases y funciones. Las partes comunes de la interfaz (cabecera, pie y formulario) se incluyen desde plantillas/.

Tabla 4

Estructura de carpetas del proyecto

Carpeta	Contenido	Pública
public/	Páginas: index, producto, admin, nuevo, editar, eliminar, foto; hoja de estilos	Sí
src/	arranque, config, funciones, validacion, fotos	No
src/Modelo/	Clases Producto, Categoria, Catalogo y enumeración Talla	No
plantillas/	cabecera.php, pie.php y formulario.php	No
datos/	categorias.json, productos.json, productos.inicial.json y fotos/	No

Nota. Elaboración propia. Solo public/ es accesible por URL.

Paso 4. Modelo orientado a objetos

Categoria y Producto son clases readonly cuyas propiedades se declaran por promoción en el constructor. El constructor de Producto verifica las reglas que siempre deben cumplirse —código con formato AAA-999, precio mayor que cero, stock no negativo y al menos una talla— y lanza InvalidArgumentException si alguna falla. Como el objeto es inmutable, editar una prenda significa crear una copia con el método con(), que recibe parámetros nombrados opcionales (Figura 1). Las tallas se modelan con la enumeración Talla, lo que impide valores inválidos como «XXXL».

Figura 1

Constructor de la clase Producto con promoción de propiedades

```
final readonly class Producto
{
    public function __construct(
        public string $codigo,
        public string $nombre,
        public Categoria $categoria,
        public float $precio,
        public int $stock = 0,
        public array $tallas = [Talla::UNICA],
        public string $color = '',
        public string $descripcion = '',
        public ?string $foto = null,
    ) { /* reglas que siempre deben cumplirse */ }
}
// Uso con parámetros nombrados: solo cambia lo indicado
$rebajado = $producto->con(precio: 79.90, stock: 10);
```

Nota. Fragmento de src/Modelo/Producto.php. Los valores por defecto permiten crear una prenda indicando solo los datos obligatorios.

Catalogo concentra la lectura y escritura de los archivos JSON. Al construirse, convierte cada fila en un objeto Categoria o Producto y ofrece los métodos agregar(), reemplazar(), eliminar(), buscar() y filtrar(), este último con parámetros nombrados y valores por defecto, por ejemplo filtrar(texto: 'jean', talla: Talla::XL, orden: 'precio-asc'). El método guardar() escribe con bloqueo exclusivo (LOCK_EX) en un archivo temporal y luego lo renombra, de modo que el

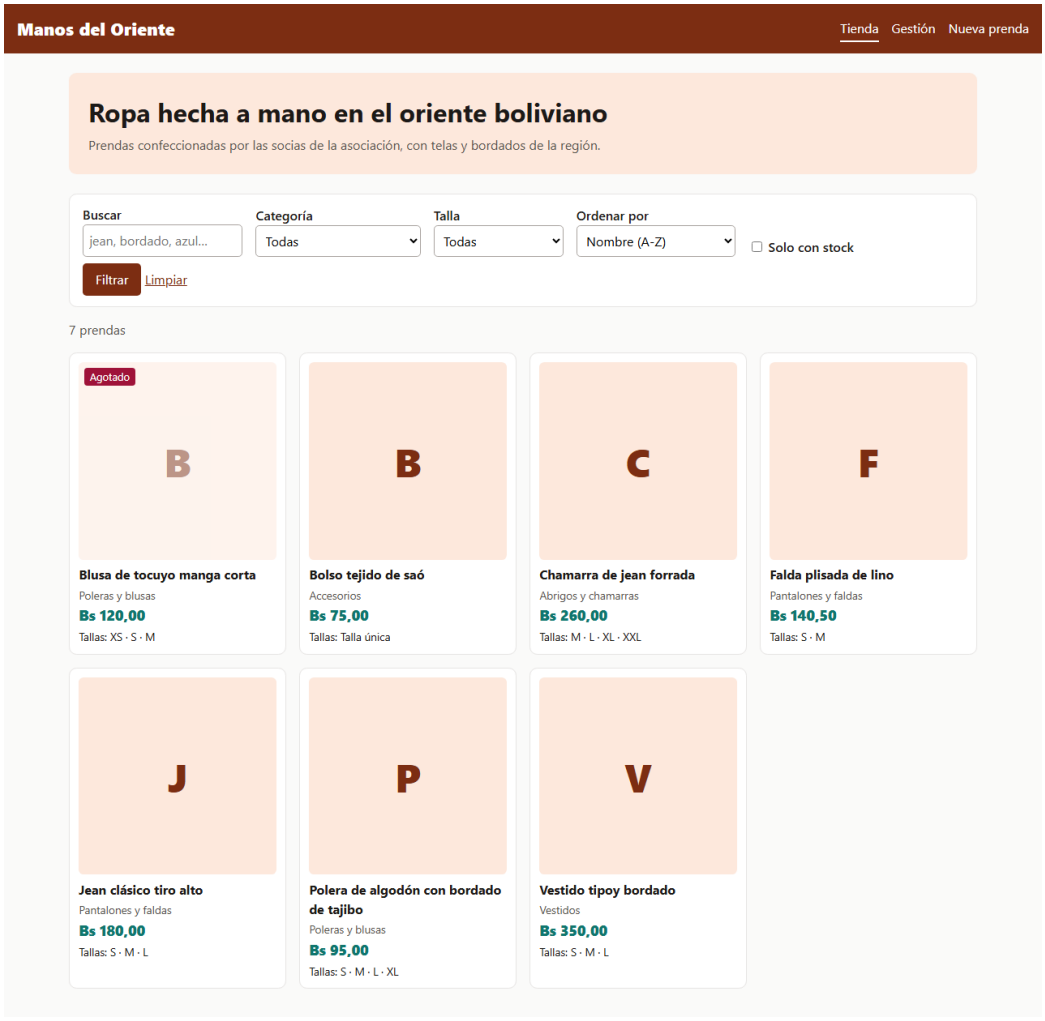
JSON nunca queda escrito a medias (RNF-04). Al concentrar la persistencia en una sola clase, la futura migración a MySQL solo requerirá reemplazar Catalogo.

Paso 5. Interfaz y formularios

La vitrina (Figura 2) muestra las prendas en una grilla adaptable con su categoría, el precio en formato boliviano (Bs 1.234,50), las tallas y la insignia «Agotado». Los filtros usan un formulario GET, por lo que cada búsqueda produce una URL que se puede compartir; los valores no válidos en la URL, como una categoría inexistente, se ignoran.

Figura 2

Vitrina de la tienda con los datos de demostración

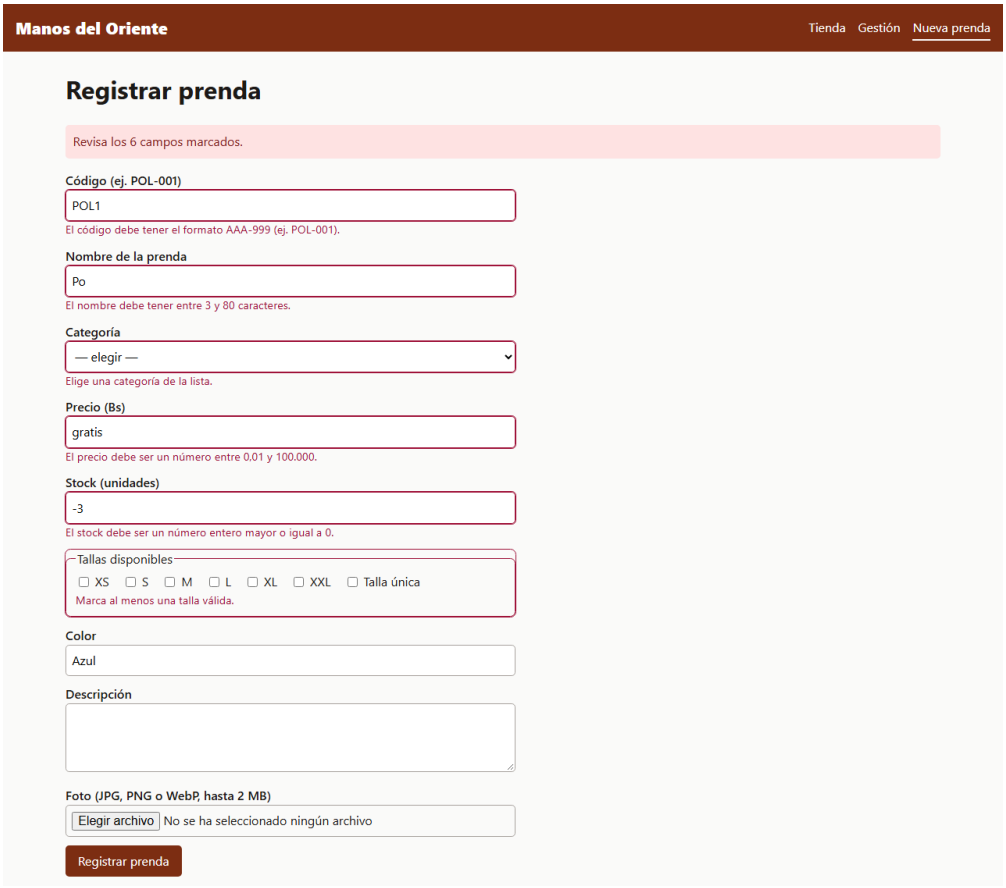


Nota. Captura del servidor embebido con los productos de demostración. Las prendas sin foto muestran la inicial del nombre.

El formulario de registro y edición usa POST con enctype="multipart/form-data". La función validar_producto() recorta espacios, convierte el código a mayúsculas, acepta la coma decimal boliviana («1.250,50»), elimina etiquetas HTML de la descripción y verifica cada campo. Si hay errores, la página responde con el código 422, conserva los valores escritos —escapados con htmlspecialchars— y muestra el mensaje junto a cada campo, enlazado con aria-describedby para los lectores de pantalla (Figura 3).

Figura 3

Formulario con errores de validación por campo



Nota. Respuesta 422 a un envío con código, nombre, categoría, precio, stock y tallas inválidos.

Paso 6. Subida segura de la foto y seguridad adicional

La función guardar_foto() aplica las recomendaciones de OWASP Foundation (s.f.-a): obtiene el tipo real con finfo y lo confirma con getimagesize() en lugar de confiar en la extensión o en el tipo enviado por el navegador; limita el tamaño a 2 MB; genera un nombre aleatorio de 24 caracteres hexadecimales con random_bytes(); y guarda el archivo en datos/fotos/, fuera de la carpeta pública. Las fotos se entregan con foto.php, que solo acepta nombres con el formato generado —lo que bloquea rutas como ../productos.json— y envía la cabecera X-Content-Type-Options: nosniff. Si otro campo del formulario falla, la foto recién subida se borra para no dejar archivos huérfanos, y al reemplazarla en una edición se elimina la anterior.

Como adelanto del proyecto final se incorporó un token CSRF de 64 caracteres hexadecimales por sesión, comparado con hash_equals(); la eliminación solo se acepta por POST (un GET responde 405 con la cabecera Allow: POST); toda salida se escapa con la función e(); y se envían las cabeceras X-Frame-Options y Referrer-Policy. La página de gestión (Figura 4) reúne las acciones de edición y eliminación y el valor del inventario.

Figura 4

Página de gestión del catálogo



Nota. El valor del inventario coincide con el cálculo manual de la Tabla 6.

Paso 7. Pruebas automáticas

Las pruebas se escribieron en verificar.php con el motor Verificador.php de la asignatura. La primera parte prueba las clases directamente; la segunda levanta el servidor embebido y envía peticiones HTTP reales con cURL, incluidos archivos maliciosos (Tabla 5).

Tabla 5

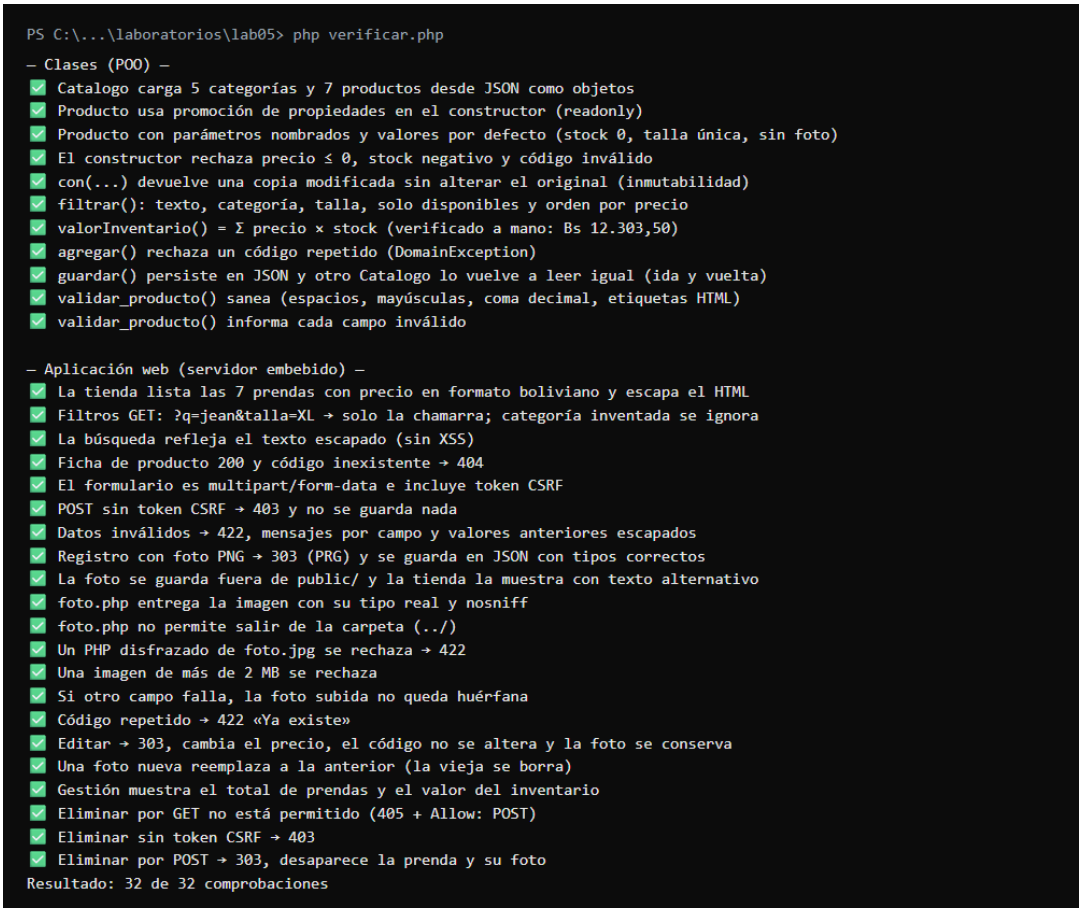
Resumen de las comprobaciones automáticas

Grupo	**Qué se comprueba**	**N.º**
Clases (POO)	Carga desde JSON, promoción de propiedades, valores por defecto, reglas del constructor, inmutabilidad, filtros, inventario, persistencia de ida y vuelta, validación, saneamiento y punto de miles	12
Vitrina y filtros	Listado, formato de precios, filtros GET, escape de la búsqueda, ficha y error 404	4
Formularios	multipart, token CSRF, error 403 sin token, error 422 con mensajes, registro con PRG (303), código repetido	5
Fotos	Guardado fuera de public/, tipo real y nosniff, bloqueo de ../, PHP disfrazado de JPG, más de 2 MB, sin huérfanas	6
Edición y eliminación	Conserva la foto y el código, reemplaza la foto, valor del inventario, 405 por GET, 403 sin token, borrado de la foto	6
Total		**33**

Nota. Elaboración propia a partir de la salida de php verificar.php.

Figura 5

Ejecución de las pruebas automáticas



Nota. Salida de php verificar.php en PowerShell con todas las comprobaciones satisfactorias.

El valor del inventario se verificó a mano con los datos de demostración (Tabla 6). El resultado coincide con el que calcula Catalogo::valorInventario() y con el que muestra la página de gestión.

Tabla 6

Cálculo manual del valor del inventario

Código	Prenda	Precio (Bs)	Stock	Subtotal (Bs)
POL-001	Polera de algodón con bordado	95,00	24	2.280,00
POL-002	Blusa de tocuyo manga corta	120,00	0	0,00
PAN-010	Jean clásico tiro alto	180,00	15	2.700,00
PAN-011	Falda plisada de lino	140,50	7	983,50
VES-100	Vestido tipoy bordado	350,00	5	1.750,00
ABR-200	Chamarra de jean forrada	260,00	9	2.340,00
ACC-300	Bolso tejido de saó	75,00	30	2.250,00
Total			**90**	**12.303,50**

Nota. Elaboración propia con los datos de datos/productos.inicial.json.

Discusión

La decisión de diseño más relevante fue hacer inmutables las clases del modelo. Obligó a escribir el método con() para editar, pero eliminó una fuente común de errores: un objeto que cambia en un lugar del código sin que el resto lo sepa. La segunda decisión, concentrar toda la lectura y escritura del JSON en Catalogo, se tomó pensando en la Actividad 2: si la persistencia cambia, solo cambia una clase.

Sobre el uso de inteligencia artificial, la IA propuso con rapidez la estructura, las clases, las pruebas y la documentación, pero cometió dos errores que solo las pruebas revelaron. Primero, escribió dos veces un valor esperado incorrecto para el inventario (Bs 10.983,50 y Bs 14.163,50); las pruebas fallaron, el valor se recalculó a mano (Bs 12.303,50 y Bs 14.042,90) y se corrigió. Segundo, en una revisión posterior detectó que «1.250» se guardaba como Bs 1,25 porque el punto de miles boliviano se interpretaba como separador decimal; se agregó una prueba y se corrigió antes de la entrega. La lección es que los valores de los reportes deben verificarse contra los datos y no aceptarse sin revisión.

Se reconocen dos limitaciones. JSON no garantiza la integridad ante escrituras simultáneas de varias personas, aunque el bloqueo y el renombrado evitan archivos corruptos; y la gestión no tiene control de acceso, de modo que cualquiera que conozca la URL puede modificar el catálogo. Ambas se resuelven en las actividades 2 y 3.

Conclusiones

Se desarrolló la aplicación de catálogo de la tienda virtual «Manos del Oriente» en PHP 8.4 cumpliendo los requisitos de la actividad: tipado estricto, funciones tipadas con parámetros nombrados y valores por defecto, organización con require, formularios GET y POST validados y saneados, subida segura de la foto, clases con promoción de propiedades y persistencia en JSON. Las 33 comprobaciones automáticas pasaron.

El diseño inmutable de las clases y la concentración de la persistencia en Catalogo dejan el sistema listo para las siguientes etapas: migrar a MySQL con consultas preparadas, agregar usuarios y roles, incorporar pedidos y reportes, exponer una API REST y publicarlo en un hosting.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Anthropic. (2026). *Claude Code* (versión con el modelo Claude Opus 5.5) [Software]. https://claude.com/claude-code

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) data interchange format* (RFC 8259). Internet Engineering Task Force. https://doi.org/10.17487/RFC8259

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://doi.org/10.17487/RFC9110

Lockhart, J. (2015). *Modern PHP: New features and good practices*. O'Reilly Media.

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

OWASP Foundation. (s.f.-a). *File upload cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

OWASP Foundation. (s.f.-b). *Cross-site request forgery prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

The PHP Group. (s.f.-a). Constructores y destructores. En *Manual de PHP*. https://www.php.net/manual/es/language.oop5.decon.php

The PHP Group. (s.f.-b). Enumeraciones. En *Manual de PHP*. https://www.php.net/manual/es/language.enumerations.php

Anexos

Anexo A. Registro de uso de inteligencia artificial

Se utilizó Claude Code (Anthropic, 2026), con acceso al repositorio, en octubre de 2026. La IA intervino en la estructura del proyecto, las clases, las pruebas y la documentación; el equipo definió el caso, verificó cada resultado y asume la responsabilidad del contenido. La versión completa es docs/BITACORA_IA.md.

Tabla 7

Registro de uso de IA en la Actividad 1

#	Pedido del equipo	Propuesta de la IA	Verificación	Decisión
1	Tienda de ropa virtual para el proyecto final	Estructura public/ src/ plantillas/ datos/ y caso «Manos del Oriente»	Comparación con la estructura de lab04	Aceptada
2	Clases con promoción de propiedades	Clases readonly, enum Talla y método con()	ReflectionMethod::isPromoted() en las pruebas	Aceptada
3	Validación y saneamiento	validar_producto() con coma decimal boliviana y strip_tags	Pruebas de entradas inválidas (422)	Aceptada
4	Subida segura de la foto	Lógica de lab04: finfo, 2 MB, nombre aleatorio, fuera de public/	PHP disfrazado de JPG, 2,2 MB y ../	Aceptada
5	Pruebas automáticas	verificar.php con datos en carpeta temporal	php verificar.php	Aceptada
6	Valor esperado del inventario	Bs 10.983,50 y Bs 14.163,50	La prueba falló; cálculo manual	Corregida
7	Informe APA y presentación	Borrador del informe y de la presentación	Revisión de cada página	Modificada
8	Revisión del estado del proyecto	Detectó que «1.250» se guardaba como Bs 1,25	Prueba nueva: «1.250» = 1250	Aceptada

Nota. Elaboración propia. El equipo debe completar su reflexión en la bitácora.

Anexo B. Matriz de trazabilidad de la Actividad 1

Tabla 8

Matriz de trazabilidad: requisito, issue, implementación y prueba

Req.	Issue	Implementación	Prueba que lo verifica
RF-01	#6	public/index.php, Catalogo::filtrar()	Listado y formato de precios
RF-02	#6	Formulario GET, filtrar() con parámetros nombrados	Filtros GET y escape de la búsqueda
RF-03	#6	public/producto.php	Ficha y error 404
RF-04	#2, #6	nuevo.php, editar.php, eliminar.php, Producto::con()	Registro (303), edición, eliminación
RF-05	#6	Catalogo::valorInventario()	Inventario verificado a mano (Tabla 6)
RNF-01	#4	src/validacion.php, función e()	422 con mensajes; saneamiento
RNF-02	#5	src/fotos.php, public/foto.php	6 pruebas de fotos
RNF-03	#6	Token CSRF con hash_equals()	403 sin token; 405 por GET
RNF-04	#3	Catalogo::guardar() con LOCK_EX y renombrado	Persistencia de ida y vuelta
RNF-05	#6	CSS adaptable, aria-describedby	Revisión manual en el navegador

Nota. Elaboración propia. Cobertura de trazabilidad: 10 de 10 requisitos (100 %).

Anexo C. Glosario

CSRF: falsificación de solicitudes entre sitios; se previene con un token secreto por sesión.

Inmutable: objeto cuyo estado no cambia después de construirse.

JSON: formato de texto liviano para el intercambio de datos.

Promoción de propiedades: declaración de las propiedades de una clase en los parámetros del constructor.

PRG: patrón Post/Redirect/Get: tras un POST exitoso se redirige con 303 para evitar reenvíos.

Saneamiento: normalización de una entrada, por ejemplo quitar espacios o etiquetas HTML.